Improving scRNA-seq clustering using Deep Learning

Eshaan Naga Venkata

IMS23097
Junior Year in Data Science

Background

Single-cell RNA sequencing (**scRNA-seq**) has revolutionised biology by enabling gene expression measurements at the single-cell level. However, its high dimensionality and sparse, count-based nature make it difficult to apply computational methods for further study. Clustering, with its ability to identify cell groups, has become an essential step in many analysis tasks such as cell type identification, trajectory inference, and differential expression analysis.

Among the many possible ways to perform clustering on gene-count data, Seruat's graph-based approach, proposed by the Satija Lab, has become very popular among researchers. It uses a modularity-based community detection algorithm to identify transcriptionally distinct cell populations. It begins with preprocessing, where it log-normalises and selects Highly Variable Genes (**HVGs**). Principal Component Analysis (**PCA**) is then used to produce a denoised embedding for constructing a K-Nearest Neighbour (**KNN**) graph. This graph is further refined using Shared Nearest Neighbour (**SNN**) similarity, to emphasise local manifold structure and down-weight distant transcriptomic relationships. Finally, the **Leiden** algorithm optimises modularity to partition cells into clusters.

Objective

Variational Autoencoders (**VAE**), a deep learning paradigm for dimensionality reduction, offer a better representation than **PCA** because they can capture non-linear relationships in the data. This study provides a comparative analysis of PCA & VAE embeddings within Seurat's workflow.

Results

0.1 Datasets and Evaluation Metric

The analysis is grounded in two cornerstone datasets of the single-cell community: Peripheral Blood Mononuclear Cells (**PBMC**) and Bone Marrow Mononuclear Cells (**BMMC**). The PBMC dataset, comprising 3,000, provides a profile of mature immune lineages, including T cells, B cells, and Monocytes. In contrast, the BMMC dataset represents a significantly more complex transcriptomic manifold, capturing the developmental hierarchy of the bone marrow, including hematopoietic stem cells and transient progenitor populations.

To quantitatively assess clustering performance, we utilise the Adjusted Rand Index (**ARI**). The ARI serves as a robust metric for cluster validation by measuring the similarity between the algorithmically derived partitions and the ground-truth biological annotations. Formally, it adjusts the standard Rand Index for chance, where a value of 1.0 represents an identical mapping to the ground truth and values near 0.0 indicate a stochastic distribution of labels. This metric is essential for penalising both false splits and improper merges within the high-dimensional latent space.

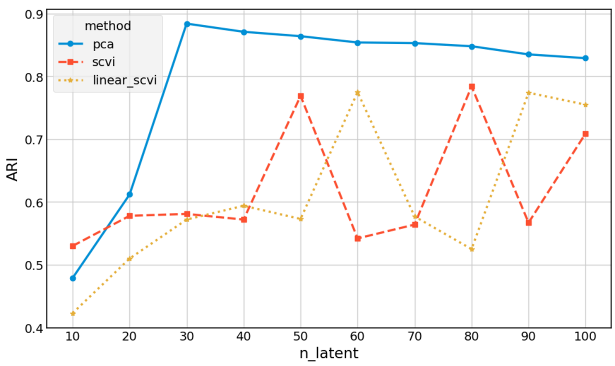
0.2 Autoencoder Models

We compare the performance of standard Principal Component Analysis (PCA) against two deep learning paradigms: **scVI** and **linear scVI** (LDVAE). Standard scVI utilizes a Variational Autoencoder (VAE) architecture with non-linear encoder-decoder networks, employing a Zero-Inflated Negative Binomial (**ZINB**) to model the technical noise and sparsity inherent in RNA-seq data. Linear scVI, conversely, constrains the decoder to a linear transformation. This architectural choice aims to combine the probabilistic advantages of the VAE, such as library size normalization and batch correction, with the interpretability and global structure preservation typically associated with PCA.

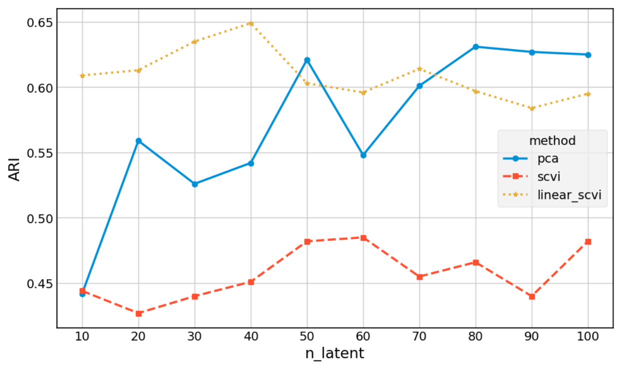
0.3 Interpretation

The results depicted in Fig. 1 reveal a compelling divergence in model performance based on tissue type. In the PBMC dataset (Fig. a), PCA demonstrates superior performance and remarkable stability, achieving a peak ARI of approximately 0.88 at 30 components. This suggests that for circulating immune populations, the transcriptomic variance is predominantly linear and effectively captured by traditional eigenvectors. Interestingly, both scVI and linear scVI exhibit significant performance fluctuations and a lower overall ARI baseline (0.55–0.78). The "zigzag" pattern observed in these lines suggests that the stochastic nature of the VAE optimization, specifically the trade-off between the reconstruction loss and the Kullback-Leibler (KL) divergence, may lead to inconsistent latent embeddings in smaller, less complex datasets.

The BMMC dataset (Fig. b) presents a shift in the hierarchy of performance. Here, linear scVI emerges as the optimal model, outperforming both PCA and non-linear scVI with a maximum ARI of 0.65 at 40 latent dimensions. This indicates that while the bone marrow manifold benefits from the probabilistic count modeling of scVI, the additional complexity of a deep, non-linear decoder may lead to overfitting or the distortion of local neighborhoods. This is corroborated by the provided UMAP visualizations in Fig. 2, which demonstrates that linear scVI effectively resolves 22 distinct clusters. These clusters represent the nuanced differentiation stages of hematopoietic cells, which standard PCA fails to separate with the same degree of clarity.



(a) PBMC Dataset



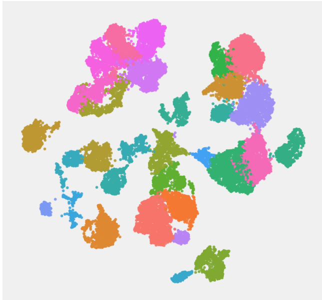
(b) BMMC Dataset

Figure 1: Latent Dimensions/PC Components vs ARI

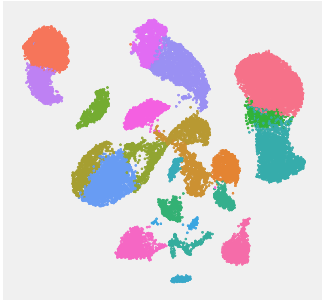
Discussion

The performance differential between linear PCA and deep generative VAEs highlights a critical trade-off in single-cell analysis. In smaller, relatively homogeneous datasets like PBMC, PCA consistently provides superior stability, yielding ARI scores near 0.88, while VAEs often suffer from overfitting on limited samples. Conversely, in larger, more complex datasets such as BMMC, VAE-based models tend to outperform PCA by effectively capturing non-linear trajectories, particularly when latent dimensions are constrained to fewer than 50. In this low-dimensional regime, VAEs provide a denoised

(a) Dim: 20, Clusters: 25

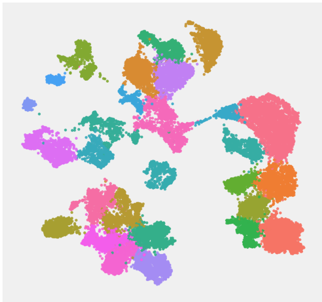


(b) Dim: 20, Clusters: 31



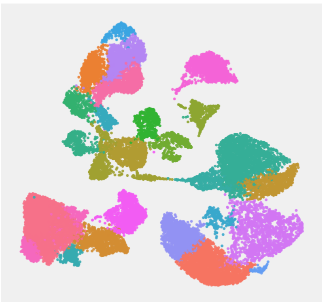
(c) Dim: 20, Clusters: 23

(d) Dim: 40, Clusters: 27

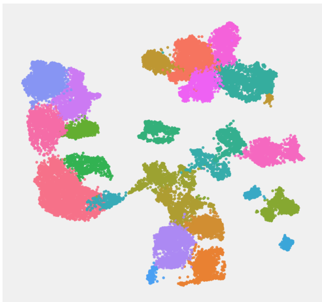


(e) Dim: 40, Clusters: 28

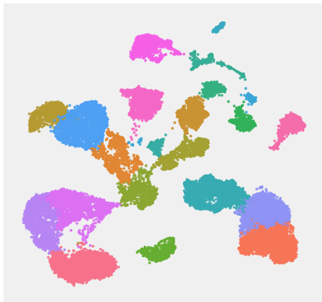
(f) Dim: 40, Clusters: 22



(g) Dim: 60, Clusters: 26

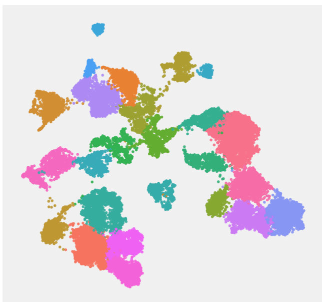


(h) Dim: 60, Clusters: 25

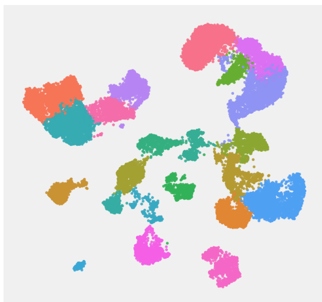


(i) Dim: 60, Clusters: 22

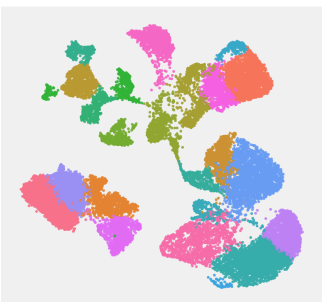
(j) Dim: 80, Clusters: 23



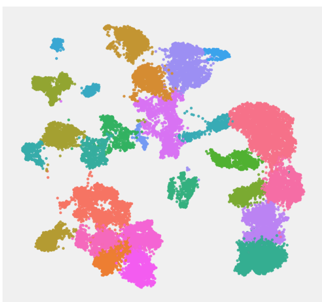
(k) Dim: 80, Clusters: 25



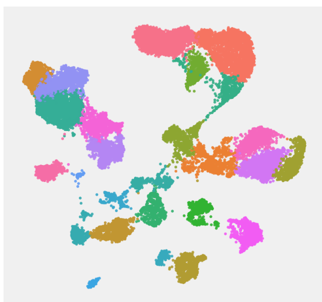
(l) Dim: 80, Clusters: 22



(m) Dim: 100, Clusters: 23



(n) Dim: 100, Clusters: 27



(o) Dim: 100, Clusters: 26

Figure 2: UMAP Visualisation of BMMC clusters. PCA(**Left**), scVI(**Middle**), Linear scVI(**Right**)

biological bottleneck that preserves subtle cell states, whereas PCA begins to incorporate technical noise as the component count increases.

Current state-of-the-art deep learning models, including **scGNN**, **scICE**, and **scAMAC**, leverage advanced architectures, such as graph neural networks and multi-scale attention, to achieve ARI scores in the **0.8–0.9** range. These models achieve high fidelity by modelling higher-order cellular interactions and utilising self-supervised learning to mitigate sparsity. This study introduces a previously unexplored approach to the existing Seurat framework by systematically benchmarking the interaction between manifold latent embeddings and graph-based community detection, opening the possibility of extending research into hybrid approaches.

The code for the above experiments is available at https://github.com/nveshaan/scRNA_cluster.

Acknowledgement

AI was used to understand the source material below and to generate code for the experiments discussed above.

- Stuart, T., Butler, A., Hoffman, P., Hafemeister, C., Papalexi, E., Mauck, W. M., Hao, Y., Stoeckius, M., Smibert, P., & Satija, R. (2019). Comprehensive Integration of Single-Cell Data. Cell. https://doi.org/10.1016/j.cell.2019.05.031
- Gayoso, A., et al. (2022). scvi-tools: a library for deep probabilistic analysis of single-cell omics data. Nature Biotechnology. https://doi.org/10.1101/2021.04.28.441833

These projects and their documentation were essential references when implementing the Seurat-like workflows and integrating scVI/LinearSCVI models.